

Publicação profissional de projetos Python

GitHub Actions, versionamento semântico, Debian, Launchpad, GPG, SBOM e Sigstore

Estudo de caso: scrolllock-led-daemon Pipeline validado: release v1.4.4, em 5 de julho de 2026 Autor do projeto: Alan Santos

1. O que este guia ensina

Este guia explica como transformar um repositório Python em uma linha de publicação automatizada. O fluxo usado como estudo de caso testa o código em várias versões do Python, calcula a próxima versão a partir dos commits, atualiza os arquivos de versão, cria uma tag, monta um pacote-fonte Debian, assina esse pacote com GPG, produz um SBOM, adiciona uma atestação Sigstore, envia o pacote ao Launchpad e preserva os artefatos no GitHub.

O objetivo não é apenas “fazer o CI ficar verde”. É construir uma cadeia na qual cada etapa tenha uma responsabilidade verificável:

1. CI: prova que o código passa nos testes.
2. Versionamento: transforma o histórico de commits em uma versão coerente.
3. Empacotamento: produz os arquivos esperados pelo ecossistema Debian.
4. GPG: autentica o responsável pelo upload no Launchpad.
5. SBOM: descreve os componentes encontrados no projeto.
6. Sigstore: registra procedência vinculada à identidade do GitHub Actions.
7. Launchpad: valida o upload e compila os pacotes binários.
8. Artefatos do GitHub: preservam evidências da execução.

2. Visão geral da arquitetura

```
[text]
commit em main
|
+--> CI: Python 3.10, 3.11, 3.12 e 3.13
|   +--> lint/compilação
|   `--> pytest
|
`--> semantic-release
    +--> analisa Conventional Commits
    +--> calcula MAJOR.MINOR.PATCH
    +--> atualiza versões e changelogs
    +--> cria commit e tag vX.Y.Z
    `--> despacha release.yml na tag
        |
        +--> verifica versões
        +--> cria orig.tar.gz
        +--> gera SBOM SPDX
        +--> importa chave GPG
        +--> assina .dsc/.buildinfo/.changes
        +--> atesta .changes com Sigstore
        +--> envia com dput ao PPA
        `--> guarda artefatos no GitHub
```

Separar o processo em três workflows evita que testes, versionamento e publicação fiquem misturados. Também torna mais fácil localizar uma falha: código, cálculo da versão ou distribuição.

3. Workflow de integração contínua

O arquivo `ci.yml` roda em pushes e pull requests direcionados à branch `main`. Uma matriz executa os testes em Python 3.10, 3.11, 3.12 e 3.13.

As etapas essenciais são:

```
[yaml]
strategy:
  matrix:
    python-version: ["3.10", "3.11", "3.12", "3.13"]

- name: Install dependencies
  run: pip install -e ".[test]" build

- name: Lint
  run: PYTHONPATH=src make lint

- name: Test
  run: PYTHONPATH=src make test
```

Por que instalar `. [test]`

Instalar somente `pytest` não instala as dependências declaradas pelo projeto. A expressão `. [test]` instala o pacote atual, suas dependências normais e o grupo opcional de testes. Isso aproxima o CI do ambiente que um usuário realmente receberá.

Regra prática

O CI deve falhar cedo e de forma barata. Testes devem acontecer antes de criar tags, assinar pacotes ou usar serviços externos.

4. Conventional Commits e versionamento semântico

O `semantic-release` calcula a versão usando o histórico desde a última tag.

Exemplos:

Commit	Efeito típico
fix: corrige leitura do LED	PATCH: 1.4.4 → 1.4.5
feat: adiciona suporte a outro LED	MINOR: 1.4.4 → 1.5.0
feat!: altera formato da configuração	MAJOR: 1.4.4 → 2.0.0
docs: melhora instalação	Conforme as regras do projeto

No estudo de caso, `.releaserc` configura analisador de commits, notas de release, changelog, um comando de preparação e um commit automático.

O script `scripts/prepare_release.py` sincroniza:

- `pyproject.toml`;
- a constante `VERSION` da aplicação;
- `debian/changelog`.

Essa sincronização é indispensável. Uma tag `v1.5.0` apontando para metadados `1.4.4` cria um pacote incoerente e deve ser rejeitada antes do build.

Evitando um ciclo infinito

O commit automático usa `[skip ci]`. Assim, o commit de release não inicia outra rodada de versionamento.

5. Por que a workflow de release é despachada explicitamente

Tags criadas por uma workflow usando o `GITHUB_TOKEN` não disparam automaticamente outra workflow baseada apenas no evento `push`. Essa proteção evita recursão acidental.

Por isso, `version.yml` possui permissão `actions: write` e executa:

```
[bash]
gh workflow run release.yml --ref "v${NEW_VERSION}"
```

O `release.yml` aceita `workflow_dispatch`, recebe a tag como referência e faz checkout do commit correto.

Essa foi uma lição importante do pipeline: a existência da tag não prova que a workflow de publicação foi executada. Sempre confira a lista de runs no GitHub Actions.

6. Verificação de consistência da versão

Antes de construir qualquer artefato, a release compara três fontes:

```
[bash]
PYPA_VERSION=$(grep -E '^version =' pyproject.toml | cut -d'"' -f2)
DEB_VERSION=$(dpkg-parsechangelog -l debian/changelog -S Version | sed 's/-.*//')
TAG_VERSION="${{ steps.version.outputs.version }}"
```

O detalhe `-l debian/changelog` é importante. Em `dpkg-parsechangelog`, `-s` significa “since”; usá-lo como opção de arquivo faz o programa interpretar `debian/changelog` como se fosse uma versão.

O build só continua se as três versões forem iguais.

7. Construção do pacote-fonte Debian

O Launchpad recebe pacotes-fonte. Ele não espera que o GitHub envie um .deb pronto. A workflow cria:

- `orig.tar.gz`: código-fonte upstream;
- `debian.tar.xz`: metadados e regras Debian;
- `.dsc`: descrição assinada do pacote-fonte;
- `.buildinfo`: informações do ambiente de build;
- `_source.changes`: manifesto assinado do upload.

O tarball upstream deve ficar no diretório pai do checkout, seguindo a convenção do `dpkg-source`:

```
[bash]
git archive \
  --format=tar.gz \
  --prefix="projeto-$VERSION/" \
  -o "../projeto_${VERSION}.orig.tar.gz" \
  "$GITHUB_SHA"
```

Depois:

```
[bash]
dpkg-buildpackage \
  -S \
  -d \
  -sa \
  --sign-backend=gpg \
  -k"$FINGERPRINT" \
  -p"$GITHUB_WORKSPACE/scripts/gpg-ci-wrapper.sh"
```

-S gera somente fonte. -sa inclui o tarball original. --sign-backend=gpg evita que versões recentes do dpkg escolham outra interface OpenPGP incompatível com o wrapper.

8. GPG: a assinatura exigida pelo Launchpad

O Launchpad exige que o upload seja assinado por uma chave GPG associada à conta responsável pelo PPA. Essa assinatura prova quem enviou o pacote e protege a integridade de `.dsc` e `.changes`.

Secrets usados:

Secret	Conteúdo
GPG_PRIVATE_KEY	Chave privada exportada em formato ASCII armored
GPG_PASSPHRASE	Senha da chave privada
LP_LAUNCHPAD_ID	Nome da conta, por exemplo alan-n7x

Nunca coloque esses valores no YAML, em commits, logs ou mensagens. Se uma credencial for exposta, revogue-a e gere outra.

Assinatura não interativa

O runner não possui terminal gráfico. A configuração usa:

```
[text]
allow-loopback-pinentry
pinentry-mode loopback
```

O wrapper chama o GPG com `--batch`, `--yes`, `--pinentry-mode loopback` e a passphrase vinda do secret. Antes do build, a workflow realiza uma assinatura pequena de teste. Isso produz um erro claro antes de gastar tempo empacotando.

GPG do upload versus chave do repositório

São assinaturas diferentes:

1. O mantenedor assina o upload com sua chave GPG.
2. O Launchpad valida essa assinatura.
3. O Launchpad compila os .deb.
4. O Launchpad assina o repositório APT com uma chave administrada pelo próprio Launchpad.

9. SBOM e Sigstore

O Syft gera um SBOM no formato SPDX JSON:

```
[bash]
syft dir:. -o spdx-json > "../projeto_${VERSION}.sbom.spdx.json"
```

Na versão 0.80 do Syft, `syft scan dir:.` não é aceito; `scan` seria interpretado como um argumento adicional.

O Cosign cria uma assinatura e um certificado vinculados à identidade OIDC da workflow. A atestação é verificada antes do upload.

Por que o SBOM e Sigstore ficam separados

Depois que `.changes` é assinado por GPG, qualquer modificação invalida sua assinatura. Portanto:

- não anexe campos personalizados ao `.changes`;
- não altere o tarball depois de gerar os hashes Debian;
- mantenha `.sig`, `.crt` e o SBOM como artefatos separados.

Sigstore complementa GPG, mas não substitui a autenticação exigida pelo Launchpad.

10. Envio ao PPA

O destino do estudo de caso é:

```
[bash]
dput "ppa:${LP_LAUNCHPAD_ID}/tools" "$CHANGES_FILE"
```

Para outro projeto, substitua `tools` pelo nome real do PPA.

O perfil ppa do `dput` já acrescenta `~` ao caminho remoto. Portanto, o identificador deve ser informado como `alan-n7x/tools`, sem `~`. Usar `ppa:~alan-n7x/tools` produz internamente `~alan-n7x/tools` e envia para um alvo inválido.

O que “dput concluído” significa

Um exit code zero do `dput` confirma que os arquivos foram transferidos. A aceitação final é assíncrona. O Launchpad ainda pode rejeitar o upload por motivos como:

- chave GPG não associada à conta;
- versão já publicada;
- distribuição inválida;
- metadados Debian incorretos;
- problemas de política do PPA.

Confirme em três lugares:

1. etapa Upload to Launchpad PPA no GitHub Actions;
2. e-mail enviado pelo Launchpad ao mantenedor;
3. página “View package details” do PPA.

No instante em que este guia foi produzido, a workflow v1.4.4 estava totalmente verde, mas a página pública do PPA ainda mostrava zero pacotes. Isso prova a diferença entre transferência concluída e aceitação/processamento pelo Launchpad.

11. Preservação dos artefatos

actions/upload-artifact não aceita padrões contendo ... Como os arquivos Debian são criados no diretório pai, primeiro eles são copiados para uma pasta interna:

```
[bash]
mkdir -p release-artifacts
find .. -maxdepth 1 -type f \
  -name "projeto_${VERSION}*" \
  -exec cp --target-directory=release-artifacts {} +
```

Depois:

```
[yaml]
- uses: actions/upload-artifact@v4
  with:
    name: launchpad-source-${{ steps.version.outputs.version }}
    path: release-artifacts/
```

Isso permite baixar posteriormente o pacote-fonte, SBOM, certificado e assinatura Sigstore diretamente da execução.

12. Falhas encontradas e o que elas ensinam

Sintoma	Causa	Correção
módulo conventional-changelog-conventionalcommits ausente	preset não instalado	instalar explicitamente o preset
.version inexistente	semantic-release não criava o arquivo	escrever a versão em \$GITHUB_OUTPUT
tag criada, release não executada	eventos do GITHUB_TOKEN não encadeiam workflows	usar workflow_dispatch explícito
versão Debian vazia	uso de -s em vez de -l	dpkg-parsechangelog -l ...
Syft recebeu dois argumentos	sintaxe incompatível com versão 0.80	syft dir:.
GPG pediu entrada em batch mode	passphrase não fornecida por loopback	secret e wrapper GPG
dput enviou para ~/usuario/PPA	~ duplicado no atalho ppa:	usar ppa:usuario/PPA
upload-artifact rejeitou ..	restrição de segurança da action	copiar para pasta no workspace

Uma pipeline madura nasce desse tipo de observação. Cada correção deve transformar um erro obscuro em uma validação antecipada e compreensível.

13. Como reutilizar esta arquitetura

Para transformar este repositório em um template:

1. Troque o nome do pacote em pyproject.toml e em debian/.

2. Atualize o mantenedor, homepage e descrição em `debian/control`.
3. Adapte `scripts/prepare_release.py` aos arquivos que guardam a versão.
4. Atualize o prefixo usado em `git archive` e na busca dos artefatos.
5. Troque o nome do PPA no comando `dput`.
6. Cadastre a chave pública GPG na conta Launchpad.
7. Configure os três secrets necessários.
8. Faça um commit convencional em uma branch de teste.
9. Valide primeiro com um PPA separado antes de publicar para usuários.

Melhorias recomendadas para um template futuro

- fixar actions e ferramentas por versão ou hash revisado;
- atualizar actions que ainda usam runtimes Node antigos;
- adicionar Ruff e MyPy somente se o projeto estiver configurado para eles;
- publicar também no PyPI e GitHub Releases em jobs independentes;
- usar environments do GitHub com aprovação para produção;
- definir retenção e checksums dos artefatos;
- testar instalação do `.deb` em contêiner Ubuntu;
- monitorar o status do upload no Launchpad após o `dput`.

14. Checklist operacional

Antes da primeira release

- PPA criado no Launchpad.
- Chave pública GPG cadastrada e confirmada no Launchpad.
- `GPG_PRIVATE_KEY` configurado no GitHub.
- `GPG_PASSPHRASE` configurado no GitHub.
- `LP_LAUNCHPAD_ID` configurado sem `~` e sem URL.
- Versões coerentes entre Python, Debian e tag.
- CI verde em todas as versões suportadas do Python.

Em cada release

- Commits seguem Conventional Commits.
- `semantic-release` criou a versão esperada.
- `release` foi executada na tag correta.
- teste GPG passou.
- `.changes` foi verificado.
- Sigstore foi verificado sem alterar `.changes`.
- `dput` terminou com sucesso.
- artefatos foram preservados.
- Launchpad enviou confirmação de aceitação.

■ pacote aparece como pending, building ou published no PPA.

15. Resultado validado no estudo de caso

Na execução v1.4.4, todas as etapas do job build-and-verify concluíram com sucesso:

- instalação das ferramentas Debian;
- instalação de Cosign e Syft;
- extração e validação da versão;
- criação do tarball e SBOM;
- importação da chave GPG;
- build e assinatura do pacote-fonte;
- criação e verificação da atestação Sigstore;
- envio ao PPA;
- coleta e upload dos artefatos;
- limpeza final.

Execução verificada:

<https://github.com/alan-n7x/scrolllock-led-daemon/actions/runs/28731681319>

PPA configurado: <https://launchpad.net/~alan-n7x/+archive/ubuntu/tools>

Conclusão

Uma publicação profissional não depende de uma única tecnologia. Ela combina testes, política de versões, formatos de pacote, identidade criptográfica, procedência e observabilidade.

O ponto mais importante é manter as garantias separadas:

- testes dizem que o software se comporta como esperado;
- versionamento diz o que mudou;
- GPG autentica o upload Debian;
- SBOM descreve componentes;
- Sigstore registra procedência da automação;
- Launchpad constrói e distribui;
- artefatos e logs permitem auditoria.

Quando cada peça tem uma responsabilidade clara, o pipeline deixa de ser uma sequência frágil de comandos e se torna uma cadeia de publicação compreensível, verificável e reutilizável.